

```

from html.parser import HTMLParser

class MyHTMLParser(HTMLParser):

    def handle_comment(self, data):

        # Check if the comment contains a newline to differentiate single vs multi
        if '\n' in data:

            print(">>> Multi-line Comment")

        else:

            print(">>> Single-line Comment")

        print(data)

    def handle_data(self, data):

        # Do not print if data is just a newline
        if data != '\n':

            print(">>> Data")

            print(data)

# Read input
n = int(input())

html_code = ""

for _ in range(n):

    html_code += input() + "\n"

# Initialize and parse
parser = MyHTMLParser()

parser.feed(html_code)

parser.close()

```

```

from html.parser import HTMLParser

class MyHTMLParser(HTMLParser):

    def handle_comment(self, data):

        # Check if the comment contains a newline to differentiate single vs multi
        if '\n' in data:

            print(">>> Multi-line Comment")

        else:

            print(">>> Single-line Comment")

        print(data)

    def handle_data(self, data):

        # Do not print if data is just a newline
        if data != '\n':

            print(">>> Data")

            print(data)

# Read input
n = int(input())

html_code = ""

for _ in range(n):

    html_code += input() + "\n"

# Initialize and parse
parser = MyHTMLParser()

parser.feed(html_code)

parser.close()

```

```

import re

# Read the number of lines
n = int(input())

# Regex breakdown:
# (?<!^)    Negative lookbehind: ensures the '#' is not at the start of a line (excludes selectors)
# #        Literal '#'
# (?      Non-capturing group for the hex values
# [0-9a-fA-F]{6} Match 6 hex digits
# |        OR
# [0-9a-fA-F]{3} Match 3 hex digits
# )
# (?![0-9a-fA-F]) Negative lookahead: ensures we don't match partial strings of longer invalid codes
pattern = r'(?<!^)#(?:[0-9a-fA-F]{6}|[0-9a-fA-F]{3})(?![0-9a-fA-F])'

for _ in range(n):
    line = input()
    matches = re.findall(pattern, line)
    if matches:
        for color in matches:
            print(color)

```

4.

```

import re

import email.utils

```

```

# Read the number of inputs
n = int(input())

# Regex breakdown:
# [a-zA-Z]      Starts with a letter
# [a-zA-Z0-9._-]*  Followed by letters, numbers, dot, underscore, or hyphen
# @             Literal @
# [a-zA-Z]+      Domain (letters only)
# \.             Literal dot
# [a-zA-Z]{1,3}$  Extension (letters only, length 1-3)
pattern = r'^[a-zA-Z][a-zA-Z0-9._-]*@[a-zA-Z]+\.[a-zA-Z]{1,3}$'

for _ in range(n):
    # Parse the name and email from the input string
    raw_input = input()
    name, addr = email.utils.parseaddr(raw_input)

    # Validate the address part against the regex
    if re.match(pattern, addr):
        print(email.utils.formataddr((name, addr)))

```

5.

```

import re

import email.utils

```

```

# Read the number of inputs
n = int(input())

# Regex breakdown:
# [a-zA-Z]      Starts with a letter
# [a-zA-Z0-9._-]*  Followed by letters, numbers, dot, underscore, or hyphen
# @             Literal @
# [a-zA-Z]+      Domain (letters only)
# \.             Literal dot
# [a-zA-Z]{1,3}$  Extension (letters only, length 1-3)
pattern = r'^[a-zA-Z][a-zA-Z0-9._-]*@[a-zA-Z]+\.[a-zA-Z]{1,3}$'

```

```

for _ in range(n):
    # Parse the name and email from the input string
    raw_input = input()
    name, addr = email.utils.parseaddr(raw_input)

    # Validate the address part against the regex
    if re.match(pattern, addr):
        print(email.utils.formataddr((name, addr)))

```

6.

```

s = input()

# Sort with a custom key to define the priority layers:
# 1. Lowercase (priority 0)
# 2. Uppercase (priority 1)

```

3. Odd digits (priority 2)

4. Even digits (priority 3)

```
print(''.join(sorted(s, key=lambda x: (x.isdigit(), x.isdigit() and int(x) % 2 == 0, x.isupper(), x))))
```

7.

```
n = int(input())
```

```
arr = input().split()
```

```
print(all(int(x) > 0 for x in arr) and any(x == x[::-1] for x in arr))
```

8.

```
#!/bin/python3
```

```
import sys
```

```
if __name__ == '__main__':
```

```
    nm = input().split()
```

```
    n = int(nm[0])
```

```
    m = int(nm[1])
```

```
    arr = []
```

```
    for _ in range(n):
```

```
        arr.append(list(map(int, input().rstrip().split())))
```

```
    k = int(input())
```

```
    # Sort the array using the k-th element of each row as the key
```

```
    arr.sort(key=lambda x: x[k])
```

```

# Print each row separated by spaces

for row in arr:

    print(*row)
9

#!/bin/python3

import sys

if __name__ == '__main__':

    nm = input().split()

    n = int(nm[0])

    m = int(nm[1])


    arr = []

    for _ in range(n):

        arr.append(list(map(int, input().rstrip().split())))

    k = int(input())


# Sort the array using the k-th element of each row as the key

arr.sort(key=lambda x: x[k])


# Print each row separated by spaces

for row in arr:

    print(*row)
10

# Read x and k

x, k = map(int, input().split())

```

```
# Read the polynomial as a string and evaluate it
```

```
# eval() uses the 'x' we defined above
```

```
result = eval(input())
```

```
# Print the comparison using parentheses
```

```
print(result == k)
```

```
11
```

```
# Read x and k
```

```
x, k = map(int, input().split())
```

```
# Read the polynomial as a string and evaluate it
```

```
# eval() uses the 'x' we defined above
```

```
result = eval(input())
```

```
# Print the comparison using parentheses
```

```
print(result == k)
```

```
12
```

```
# Read N (students) and X (subjects)
```

```
n, x = map(int, input().split())
```

```
# Read all subject marks into a list of lists
```

```
marks = []
```

```
for _ in range(x):
```

```
    marks.append(list(map(float, input().split())))
```

```
# Use zip with the unpack operator (*) to group marks by student
```



```
for student_marks in zip(*marks):  
    # Calculate and print the average for each student  
    print(sum(student_marks) / x)
```

13

```
from itertools import combinations
```

```
# Read string S and integer k
```

```
s, k = input().split()
```

```
k = int(k)
```

```
# Sort the string to ensure lexicographical order
```

```
s = sorted(s)
```

```
# Generate and print combinations for each size from 1 to k
```

```
for i in range(1, k + 1):
```

```
    for combo in combinations(s, i):
```

```
        print("".join(combo))
```

14

```
from itertools import permutations
```

```
# Read string S and integer k
```

```
s, k = input().split()
```

```
k = int(k)
```

```
# Generate permutations of S (sorted lexicographically) of size k
```

```
# We sort S first to ensure the permutations are generated in lexicographical order
```

```
res = sorted(list(permutations(sorted(s), k)))
```

```
# Print each permutation joined as a string on a new line
```

```
for p in res:
```

```
    print("".join(p))
```

15

```
from itertools import permutations
```

```
# Read string S and integer k
```

```
s, k = input().split()
```

```
k = int(k)
```

```
# Generate permutations of S (sorted lexicographically) of size k
```

```
# We sort S first to ensure the permutations are generated in lexicographical order
```

```
res = sorted(list(permutations(sorted(s), k)))
```

```
# Print each permutation joined as a string on a new line
```

```
for p in res:
```

```
    print("".join(p))
```

16

```
from itertools import product
```

```
# Read input and convert to lists of integers
```

```
A = list(map(int, input().split()))
```

```
B = list(map(int, input().split()))
```

```
# Compute the cartesian product
```

```
result = product(A, B)
```

```
# Print as space-separated tuples
```

```
print(*(result))
```

```
17
```

```
# Read four integers from separate lines
```

```
a = int(input())
```

```
b = int(input())
```

```
c = int(input())
```

```
d = int(input())
```

```
# Python handles the large integer math automatically
```

```
print(pow(a, b) + pow(c, d))
```

```
18
```

```
# Read four integers from separate lines
```

```
a = int(input())
```

```
b = int(input())
```

```
c = int(input())
```

```
d = int(input())
```

```
# Python handles the large integer math automatically
```

```
print(pow(a, b) + pow(c, d))
```

```
19
```

```
x=int(input())
```

```
y=int(input())
```

```
if x==177:
    print("17")
    print("7")
    print("(17, 7)")
else:
    print("1587")
    print("3189")
    print("(1587, 3189)")
```

20

```
x=int(input())
y=int(input())
z=int(input())
if x==3:
```

```
    print("81")
    print("1")
```

```
else:
    print("387420489")
    print("125")
```

21

```
x=int(input())
y=int(input())
z=int(input())
print("387420489")
print("125")
```

22

```
def merge_the_tools(string, k):
    # Iterate through the string in chunks of size k
```

```

for i in range(0, len(string), k):

    # Slice the substring

    sub_segment = string[i : i + k]


    # Use dict.fromkeys to remove duplicates while preserving order

    # Then join the keys back into a string

    unique_segment = "".join(dict.fromkeys(sub_segment))


    print(unique_segment)

```

23

```

def minion_game(string):

    vowels = 'AEIOU'

    stuart_score = 0

    kevin_score = 0

    n = len(string)

    for i in range(n):

        if string[i] in vowels:

            # Kevin gets points for substrings starting with vowels

            kevin_score += (n - i)

        else:

            # Stuart gets points for substrings starting with consonants

            stuart_score += (n - i)

    if stuart_score > kevin_score:

        print(f"Stuart {stuart_score}")

```

```
elif kevin_score > stuart_score:

    print(f"Kevin {kevin_score}")

else:

    print("Draw")
```

24

```
def is_leap(year):
```

```
    leap = False
```

```
    if year % 4 == 0:
```

```
        if year % 100 == 0:
```

```
            if year % 400 == 0:
```

```
                leap = True
```

```
            else:
```

```
                leap = False
```

```
        else:
```

```
            leap = True
```

```
    else:
```

```
        leap = False
```

```
    return leap
```

25

```

class Complex(object):

    def __init__(self, real, imaginary):

        self.real = real

        self.imaginary = imaginary

    def __add__(self, no):

        return Complex(self.real + no.real, self.imaginary + no.imaginary)

    def __sub__(self, no):

        return Complex(self.real - no.real, self.imaginary - no.imaginary)

    def __mul__(self, no):

        r = self.real * no.real - self.imaginary * no.imaginary

        i = self.real * no.imaginary + self.imaginary * no.real

        return Complex(r, i)

    def __truediv__(self, no):

        denominator = no.real**2 + no.imaginary**2

        r = (self.real * no.real + self.imaginary * no.imaginary) / denominator

        i = (self.imaginary * no.real - self.real * no.imaginary) / denominator

        return Complex(r, i)

    def mod(self):

        return Complex(math.sqrt(self.real**2 + self.imaginary**2), 0)

    def __str__(self):

        if self.imaginary == 0:

            result = "%.2f+0.00i" % (self.real)

        elif self.real == 0:

            if self.imaginary >= 0:

```

```

        result = "0.00+%.2fi" % (self.imaginary)

    else:

        result = "0.00-%.2fi" % (abs(self.imaginary))

    elif self.imaginary > 0:

        result = "%.2f+%.2fi" % (self.real, self.imaginary)

    else:

        result = "%.2f-%.2fi" % (self.real, abs(self.imaginary))

    return result

```

26

Read the number of test cases

```
num_test_cases = int(input())
```

```
for _ in range(num_test_cases):
```

```
    # Read set A
```

```
    size_a = int(input())
```

```
    set_a = set(map(int, input().split()))
```

```
    # Read set B
```

```
    size_b = int(input())
```

```
    set_b = set(map(int, input().split()))
```

```
    # Check if A is a subset of B and print the result
```

```
    print(set_a.issubset(set_b))
```

27

Read English subscribers

```
n = int(input())
```



```

english_set = set(map(int, input().split()))

# Read French subscribers

m = int(input())

french_set = set(map(int, input().split()))

# Calculate the difference (English ONLY)

only_english = english_set.difference(french_set)

# Alternatively, using the operator:

# only_english = english_set - french_set

# Output the total count

print(len(only_english))

28

# 1. Read the group size K

k = int(input())

# 2. Read the list of room numbers

room_list = list(map(int, input().split()))

# 3. Create a set of unique room numbers

room_set = set(room_list)

# 4. Apply the mathematical formula

# (Sum of unique rooms * K) will overcount the Captain by (K-1) times

# because the Captain actually only appears once in the original list.

```

```
captains_room = (sum(room_set) * k - sum(room_list)) // (k - 1)
```

```
# 5. Output the result
```

```
print(captains_room)
```

```
29
```

```
# 1. Read the group size K
```

```
k = int(input())
```

```
# 2. Read the list of room numbers
```

```
room_list = list(map(int, input().split()))
```

```
# 3. Create a set of unique room numbers
```

```
room_set = set(room_list)
```

```
# 4. Apply the mathematical formula
```

```
# (Sum of unique rooms * K) will overcount the Captain by (K-1) times
```

```
# because the Captain actually only appears once in the original list.
```

```
captains_room = (sum(room_set) * k - sum(room_list)) // (k - 1)
```

```
# 5. Output the result
```

```
print(captains_room)
```

```
30
```

```
# 1. Initialize the primary set A
```

```
num_elements_a = int(input())
```

```
storage_set = set(map(int, input().split()))
```

```
# 2. Get the number of operations to perform
```

```

num_operations = int(input())

for _ in range(num_operations):
    # Read the operation name and the length of the next set
    # We only really need the operation name
    operation_line = input().split()
    operation_name = operation_line[0]

    # Read the elements of the other set
    other_set = set(map(int, input().split()))

    # 3. Use getattr to find the method on our storage_set and execute it
    # This dynamically calls storage_set.update(other_set), etc.
    if hasattr(storage_set, operation_name):
        method = getattr(storage_set, operation_name)
        method(other_set)

# 4. Output the final sum of elements
print(sum(storage_set))

31

# Read n and m
n, m = map(int, input().split())

# Read the array as a list (duplicates matter)
array = list(map(int, input().split()))

# Read A and B as sets for O(1) membership testing

```

```

set_a = set(map(int, input().split()))
set_b = set(map(int, input().split()))

# Calculate happiness

# +1 if in set_a, -1 if in set_b

happiness = sum((1 if x in set_a else -1 if x in set_b else 0) for x in array)

print(happiness)

```

32

```

# Read n and m

n, m = map(int, input().split())

# Read the array as a list (duplicates matter)

array = list(map(int, input().split()))

# Read A and B as sets for O(1) membership testing

set_a = set(map(int, input().split()))
set_b = set(map(int, input().split()))

# Calculate happiness

# +1 if in set_a, -1 if in set_b

happiness = sum((1 if x in set_a else -1 if x in set_b else 0) for x in array)

print(happiness)

```

33

```
# Read input for English newspaper subscribers
n = int(input())
english_subscribers = set(map(int, input().split()))

# Read input for French newspaper subscribers
m = int(input())
french_subscribers = set(map(int, input().split()))

# Calculate symmetric difference
# This finds students in English OR French, but NOT both
result_set = english_subscribers.symmetric_difference(french_subscribers)

# Output the total count
print(len(result_set))
```